IINTS-AF SDK Manual

Research Use Only — Not for Clinical Care

IINTS-AF SDK

Contents

1 Full Technical Manual

Version 1.3.2 | Python SDK

PRE-CLINICAL USE ONLY - NOT FOR PATIENT CARE

This SDK is intended for research, simulation, and algorithm validation. It has NOT received FDA clearance or CE marking for clinical use.

1.1 How to Use This Manual (Read First)

New users should first read: - `docs/PLAIN_LANGUAGE_GUIDE.md` - `README.md`

In plain words, this SDK lets you: - run insulin algorithm simulations, - enforce deterministic safety constraints, - inspect audit trails and reports.

It does not provide direct clinical dosing.

This manual is long. Use this map to find what you need fast: - Full SDK overview: `docs/COMPREHENSIVE_GUIDE.md` - CLI reference: `docs/TECHNICAL_README.md` - Research track (predictor training): `research/README.md` - Notebooks: `examples/notebooks/README.md`

Recommended notebook order (best for new users): - `00_Quickstart.ipynb` - run a full simulation - `01_Presets_and_Scenarios.ipynb` - scenarios + presets - `02_Safety_and_Supervisor.ipynb` - safety checks - `03_Audit_Trail_and_Report.ipynb` - audit trail + PDF - `04_Baseline_and_Metrics.ipynb` - metrics & baselines - `05_Devices_and_HumanInLoop.ipynb` - pumps/sensors + manual override - `06_Optional_Torch_LSTM.ipynb` - predictor training - `07_Ablation_Supervisor.ipynb` - safety ablation - `08_Data_Registry_and_Import.ipynb` - data import + registry

Quick task routing: - Run a simulation fast: Section 2.2 - Customize safety limits: Section 4.2 - Generate audit report: Section 5.4 - Train an AI predictor: Section 8.2 + `research/README.md` - Use the local AI assistant: Section 8.6 + `docs/AI_ASSISTANT.md`

Evidence routing: - Peer-reviewed source mapping: `docs/EVIDENCE_BASE.md` - CLI source manifest: `iints sources` or `iints sources --output-json results/source_manifest.json`

1.2 Table of Contents

1.2.7 7. Practical Examples

7.1 Complete Algorithm Example 7.2 CLI vs Python API Comparison 7.3 Stress Testing Patterns 7.4 Human-in-the-Loop Integration 7.5 Data Import Workflows

1.2.8 8. Advanced Topics

8.1 Commercial Pump Emulators 8.2 Dataset Registry Usage 8.3 Reproducibility Techniques 8.4 Performance Profiling 8.5 Custom Metrics Calculation 8.6 Local AI Assistant (Ministral 3 Open-Weight via Ollama)

1.2.9 9. Troubleshooting

9.1 Common Installation Issues 9.2 Simulation Problems 9.3 Algorithm Development Tips 9.4 Data Import Solutions 9.5 Performance Optimization

1.2.10 10. Quick Reference

10.1 Essential CLI Commands 10.2 Python Code Snippets 10.3 Safety Thresholds Cheatsheet 10.4 Clinical Metrics Targets

1.2.11 11. Glossary of Terms

1.3 1. Executive Summary

The IINTS-AF SDK (Intelligent Insulin Titration System for Artificial Pancreas) is a safety-first simulation and validation platform for insulin dosing algorithms targeting closed-loop insulin delivery research.

1.3.1 Key Capabilities

- Plug-and-play algorithm architecture - Implement one method, get full simulation
- 9-layer Independent Safety Supervisor - Deterministic override guarantees
- Realistic device models - CGM sensor and insulin pump error simulation
- Commercial pump emulators - Medtronic 780G, Omnipod 5, Tandem Control-IQ
- Clinical metrics - TIR, GMI, CV, LBGI, HBGI per ATTD/ADA guidelines
- Complete audit trail - JSONL + CSV + JSON summary with integrity hashing
- PDF clinical reports - Visual reports with glucose traces and safety summaries
- Benchmark mode - Head-to-head algorithm comparison
- Real-world data import - Dexcom, Libre, Nightscout, AIDE/PEDAP, AZT1D, HUPA-UCM
- Optional AI predictor - Proactive glucose forecasting
- Reproducible runs - Seeded randomness and signable manifests

1.3.2 Intended Use

This SDK is intended for: - Pre-clinical algorithm validation - Academic research - Educational purposes - Regulatory submission preparation

NOT intended for: - Direct patient care - Clinical decision-making - Medical device deployment without regulatory review

---

1.4 2. Getting Started

1.4.1 2.1 Installation Guide

1.4.1.1 Option 1: Install from PyPI (Recommended)

```
# You can run this from any working folder
python3 -m venv .venv
source .venv/bin/activate  # macOS/Linux
# .venv\Scripts\activate   # Windows

# Install SDK + MDMP support
python -m pip install -U pip
python -m pip install -U "iints-sdk-python35[mdmp]"

# Verify installation
iints doctor --smoke-run
iints --help
```

1.4.1.2 Option 2: Development Install

```
# Must be run from the repository root (the folder with pyproject.toml)
git clone https://github.com/python35/IINTS-SDK.git
cd IINTS-SDK
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -U pip
python -m pip install -U -e ".[mdmp]"
```

1.4.1.3 Option 3: With Research Extras (AI Predictor)

```
pip install iints-sdk-python35[research]
```

Folder rule

- Installed `iints ...` commands can run from any folder.
- `python -m pip install -e ".[mdmp]"` only works from the SDK repository root.
- After `iints quickstart`, switch into the generated project folder before running `iints presets run ...`.

For the detailed path guide, see `docs/INSTALLATION.md`.

1.4.2 2.2 Your First Simulation (60 seconds)

1.4.2.1 Using CLI (Quickstart)

```
# Create quickstart project
iints quickstart --project-name iints_quickstart
cd iints_quickstart

# Run clinic-safe preset
iints presets run --name baseline_t1d --algo algorithms/example_algorithm.py
```

1.4.2.2 Using Python API

```
from iints import run_simulation
from iints.core.algorithms.pid_controller import PIDController

# Run 12-hour simulation
results = run_simulation(
    algorithm=PIDController(),
    duration_minutes=720,
    seed=42
)

print(f"Results saved to: {results['results_csv']}")
print(f"Report generated: {results['clinical_report']}")
```

What this does: - Creates a virtual patient with default parameters - Runs PID controller algorithm for 12 hours - Generates results CSV, clinical report PDF, and audit trail - Compares against baseline algorithms automatically

Sanity check (first run): - `results/results.csv` exists and grows during the run - `results/clinical_reports/` contains a PDF - Console shows no safety contract violations

Expected runtime: - Laptop CPU: ~30-90 seconds for the quickstart preset - GPU not required

1.4.3 2.3 Understanding the Output Files

After running a simulation, you'll find these files in the `results/` folder:

File	Description
`results.csv`	Every simulation step with glucose, insulin, IOB, COB, safety events
`clinical_report.pdf`	Visual report with charts and clinical metrics
`config.json`	Exact configuration used (for reproducibility)
`run_metadata.json`	Run ID, seed, platform info, timestamps
`run_manifest.json`	SHA-256 hashes of all files (integrity verification)
`audit/audit_trail.csv`	Detailed per-step audit trail
`audit/safety_summary.json`	Safety interventions summary
`baseline/pid_results.csv`	PID controller baseline comparison
`baseline/standard_pump.csv`	Standard pump baseline comparison

Quick interpretation: - Start with `clinical_report.pdf` for a human-readable summary. - Use `results.csv` for plotting and deeper analysis. - Use `audit/safety_summary.json` to explain why the supervisor intervened.

Data consistency checks: - `glucose_actual_mgdl` should be in 40-400 mg/dL range. - `patient_iob_units` and `patient_cob_grams` should be >= 0. - Large jumps (>60 mg/dL in 5 min) usually indicate data issues.

Example: Loading Results in Python

```python
import pandas as pd

# Load main results
df = pd.read_csv("results/your_run/results.csv")
print(df.head())

# Load safety summary
import json
with open("results/your_run/audit/safety_summary.json") as f:
    safety = json.load(f)
print(f"Total interventions: {safety['intervention_count']}")
```

1.4.4 2.4 Creating Your First Custom Algorithm

1.4.4.1 Step 1: Generate Template

```
iints new-algo --name MyAlgorithm --output-dir algorithms/
```

1.4.4.2 Step 2: Implement Logic

```python
# algorithms/my_algorithm.py
from iints.api.base_algorithm import InsulinAlgorithm, AlgorithmInput

class MyAlgorithm(InsulinAlgorithm):
    def get_algorithm_metadata(self):
        return {
            "name": "My Custom Algorithm",
            "version": "0.1.0",
            "description": "My first insulin dosing algorithm"
        }

    def predict_insulin(self, data: AlgorithmInput) -> dict:
        # Simple example: basal rate + correction
        basal = 0.9  # U/hr
        correction = 0.0

        # Add correction if glucose is high
        if data.current_glucose > 180:
            correction = (data.current_glucose - 180) / 50  # ISF of 50

        return {
            "basal_insulin": basal,
            "bolus_insulin": correction,
            "reason": f"Basal {basal}U + correction {correction}U for glucose
            ↪ {data.current_glucose}mg/dL"
        }
```

1.4.4.3 Step 3: Test Your Algorithm

```
# Run with your custom algorithm
iints run --algo algorithms/my_algorithm.py --duration 1440
```

1.4.5 2.5 Adding Stress Tests

Add realistic challenges to test algorithm robustness:

```
from iints.core.simulator import Simulator, StressEvent

sim = Simulator(algorithm=MyAlgorithm(), patient_config="default")

# Add meal at 8:00 AM (480 minutes)
sim.add_stress_event(StressEvent(
    start_time=480,
    event_type="meal",
    value=60  # 60g carbs
))

# Add exercise at 6:00 PM (1080 minutes)
sim.add_stress_event(StressEvent(
    start_time=1080,
    event_type="exercise",
    value=30  # 30 minutes moderate exercise
))

# Add sensor noise
sim.add_stress_event(StressEvent(
    start_time=300,
    event_type="sensor_noise",
    value=15.0  # 15 mg/dL standard deviation
))

results = sim.run_batch(duration_minutes=1440)
```

Available Stress Events: - `meal`: Carbohydrate intake (value = grams) - `exercise`: Physical activity (value = minutes) - `sensor_noise`: CGM noise (value = std deviation) - `sensor_dropout`: CGM signal loss (value = probability) - `pump_failure`: Insulin delivery failure (value = probability) - `hormonal_change`: Dawn phenomenon simulation

1.4.6 2.6 Benchmarking Against Baselines

Compare your algorithm against standard approaches:

```
# CLI benchmark
iints benchmark \
  --algo-to-benchmark algorithms/my_algorithm.py \
  --patient-configs-dir src/iints/data/virtual_patients \
  --scenarios-dir scenarios \
  --output-dir results/benchmark
```

```python
# Python API benchmark
from iints.analysis.benchmark import run_benchmark

results = run_benchmark(
    algorithm_path="algorithms/my_algorithm.py",
    patient_configs=["default", "adolescent", "insulin_resistant"],
    scenarios=["baseline", "meal_challenge", "exercise_stress"],
    duration_minutes=1440
)
```

1.4.7 2.7 Using Preset Scenarios

Clinic-safe scenarios for reproducible testing:

```bash
# List available presets
iints presets list

# Run a specific preset
iints presets run --name hypo_prone_night --algo algorithms/my_algorithm.py
```

Available Presets: - `baseline_t1d`: Standard Type 1 diabetes profile - `hypo_prone_night`: Nighttime hypoglycemia risk - `hyper_challenge`: Post-meal hyperglycemia - `pizza_paradox`: Delayed carb absorption - `midnight_crash`: Nocturnal hypoglycemia - `exercise_stress`: Physical activity impact - `sensor_noise`: CGM accuracy challenges

1.4.8 2.8 Importing Real CGM Data

Test against real patient data:

```bash
# From CSV file
iints import-data --input-csv my_cgm_export.csv \
  --data-format dexcom \
  --output-dir results/imported
```

```python
# Python API
from iints.data.importer import import_cgm_csv
from iints.core.simulator import Simulator

result = import_cgm_csv(
    "my_cgm_export.csv",
    data_format="dexcom",  # or "libre", "generic"
    scenario_name="Patient A - Week 1",
)

sim = Simulator(
    algorithm=MyAlgorithm(),
    scenario=result.scenario,
)
```

Supported Formats: - Dexcom CSV export - Libre CSV export - Generic CSV (auto-detects columns)

Minimum required columns (generic CSV): - Timestamp (ISO or epoch minutes) - Glucose (mg/dL)

Optional but recommended: - Carbs (grams) - Insulin (units) - Notes/events

Common import pitfalls: - Mixed timezones or missing timezone offsets - Glucose in mmol/L (convert to mg/dL) - Duplicate timestamps (keep the latest reading)

Quick validation: - Plot glucose vs time to confirm it is smooth and within 40-400 mg/dL - Ensure meal events align with glucose rises (15-90 minutes after) - Nightscout JSON - Dataset registry packs (AIDE, PEDAP, AZT1D, HUPA-UCM)

1.4.9 2.9 Generating Custom Reports

```python
from iints.analysis.reporting import ClinicalReportGenerator

generator = ClinicalReportGenerator()
generator.generate_pdf(
    results_df,  # Your simulation results DataFrame
    safety_report,  # Safety summary dict
    "results/custom_report.pdf",
    title="My Algorithm Performance Report",
    include_trend_analysis=True,
    highlight_safety_events=True
)
```

Report Contents: - Glucose trace chart with target range - Insulin delivery chart - Clinical metrics table (TIR, GMI, CV, LBGI, HBGI) - Safety interventions summary - Top intervention reasons - Configuration overview

1.4.10 2.10 What Next?

Recommended Next Steps:

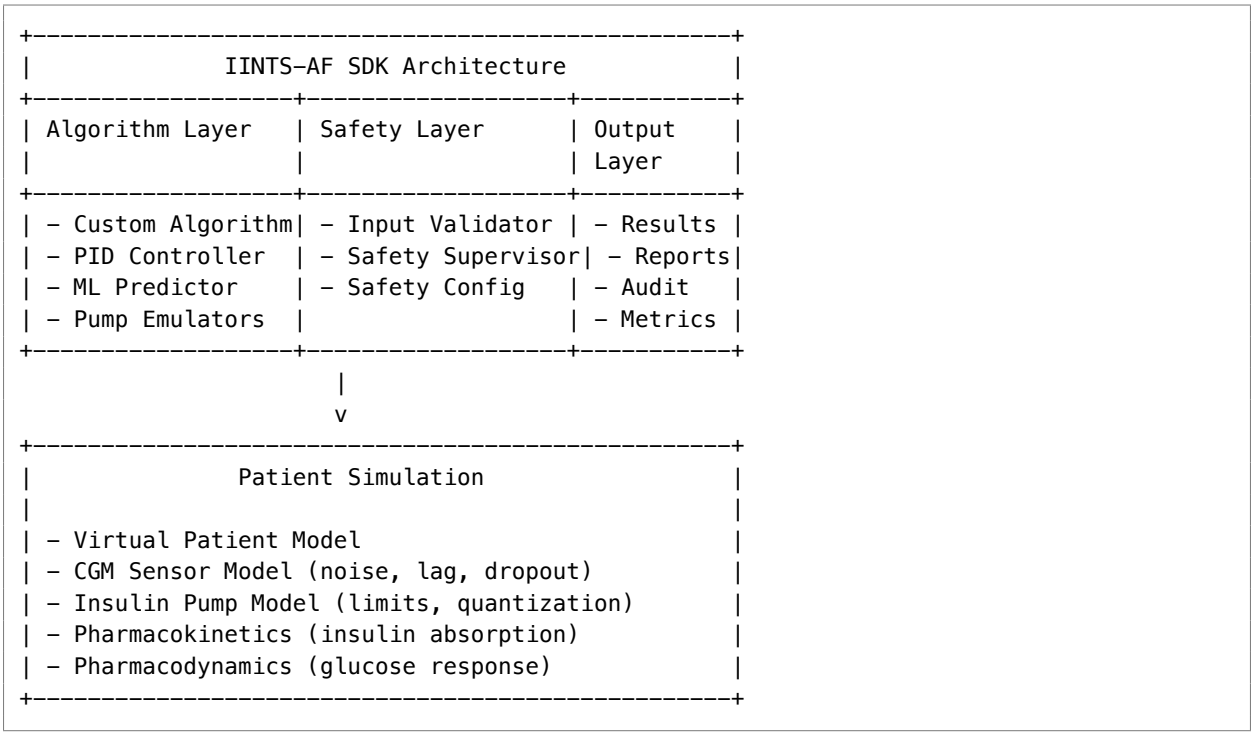
1. • Complete the Getting Started guide
2. [Metrics] Run benchmark comparisons
3. [Research] Test with stress scenarios
4. [Performance] Import real CGM data
5. [AI] Explore AI predictor integration
6. [Notes] Generate clinical reports
7. [Training] Review safety architecture
8. [Tools] Customize patient profiles
9. [Package] Package algorithm for distribution
10. [Announcement] Share results with community

Need Help? - Check Troubleshooting section (9.0) - Review FAQ (17.0) - Join community discussions - Submit issues on GitHub

---

1.5 3. Architecture Overview

1.5.1 3.1 System Components



1.5.2 3.2 Data Flow

1. Algorithm requests insulin dose
   v
2. Input Validator checks glucose values
   v
3. Safety Supervisor applies 9 safety checks
   v
4. Approved dose sent to pump model
   v
5. Pump model simulates delivery (with possible errors)
   v
6. Patient model calculates glucose impact
   v
7. CGM sensor model adds noise/lag
   v
8. New glucose reading returned to algorithm
   v
9. Audit trail logs all decisions
   v
10. Repeat every time step (default: 5 minutes)

1.5.3 3.3 Safety Layer Integration

The Independent Safety Supervisor runs deterministically and can: - - Override dangerous algorithm requests - - Log all interventions with reasons - - Enforce hard limits (hypoglycemia protection) - - Apply dynamic limits (IOB clamping) - - Validate all inputs/outputs

Key Principle: Safety layer is always active and cannot be disabled.

1.6 4. Safety Architecture (Critical Section)

1.6.1 4.1 Design Philosophy

Safety-First Principles:

1. Deterministic Overrides: Same input -> same safety decision
2. Fail-Safe Defaults: When in doubt, reduce insulin
3. Audit Everything: Every decision logged for accountability
4. Transparent Logic: Clear reasons for all interventions
5. Configurable Thresholds: Adapt to different patient profiles

Safety Guarantees: - No algorithm can deliver unsafe doses - Hypoglycemia protection is absolute (< 40 mg/dL emergency stop) - All interventions are logged and explainable - Configuration is validated before simulation starts

1.6.2 4.2 SafetyConfig Configuration

```python
from iints.core.safety import SafetyConfig

# Default configuration (clinic-safe)
config = SafetyConfig(
    # Hypoglycemia protection
    hypo_cutoff=70.0,  # mg/dL - start reducing insulin
    severe_hypo_cutoff=54.0,  # mg/dL - emergency stop
    critical_hypo_cutoff=40.0,  # mg/dL - immediate termination

    # Hyperglycemia limits
    hyper_cutoff=300.0,  # mg/dL - maximum allowed

    # Insulin limits
    max_basal_rate=2.0,  # U/hr
    max_bolus=5.0,  # U per bolus
    max_iob=10.0,  # U total active insulin

    # Rate limits
    max_insulin_per_hour=15.0,  # U/hr rolling window
    max_insulin_per_day=80.0,  # U/day absolute limit

    # Trend protection
    contract_enabled=True,
    contract_glucose_threshold=90.0,  # mg/dL
    contract_trend_threshold=-1.0,  # mg/dL per 5 minutes
)
```

When to tune SafetyConfig - Only after you can reproduce a baseline run with stable glucose and no crashes. - Increase strictness (lower cutoffs / lower max rates) when testing new or unstable algorithms. - Relax cutoffs only for controlled research experiments with full audit logs.

Recommended baseline ranges (adult research) - `max_bolus`: 2-6 U - `max_basal_rate`: 1-3 U/hr - `max_iob`: 6-12 U - `hyper_cutoff`: 250-300 mg/dL

Audit note: All SafetyConfig values are written to `run_metadata.json` and `audit/safety_summary.json`.

1.6.3 4.3 9 Safety Checks Explained

The IndependentSupervisor applies these checks in order:

```
1. Predictive Hypo Guard [EMERGENCY]
   - If glucose < 70 AND falling fast (-3+ mg/dL per 5min)
   - Action: Suspend insulin for 30 minutes
   - Rationale: Prevent imminent severe hypo

2. Basal Rate Limit [WARNING]
   - If basal > max_basal_rate
   - Action: Cap at max_basal_rate
   - Rationale: Prevent basal overdose

3. Hard Hypo Cutoff [EMERGENCY]
   - If glucose < 54 mg/dL
   - Action: Suspend all insulin
   - Rationale: Severe hypoglycemia protection

4. Severe Hypo Emergency Stop [EMERGENCY]
   - If glucose < 40 mg/dL
   - Action: Terminate simulation
   - Rationale: Critical hypoglycemia - stop everything

5. Glucose Level Clamp [CRITICAL/WARNING]
   - If glucose > 300 mg/dL
   - Action: Reduce insulin by 50%
   - Rationale: Prevent over-correction

6. Rate-of-Change Trend Stop [CRITICAL]
   - If glucose falling > 5 mg/dL per 5min
   - Action: Suspend insulin
   - Rationale: Rapid drop protection

7. Dynamic IOB Clamp [WARNING]
   - If IOB > max_iob
   - Action: Reduce dose to stay under max_iob
   - Rationale: Prevent insulin stacking

8. Bolus Stacking Check [WARNING]
   - If recent boluses > safety limit
   - Action: Delay or reduce bolus
   - Rationale: Prevent bolus overlap

9. 60-Minute Rolling Cap [WARNING]
```

```
- If insulin in last 60min > max_insulin_per_hour
- Action: Reduce dose to stay under limit
- Rationale: Hourly limit enforcement
```

1.6.4 4.4 Safety Levels

Level	Severity	Action
INFO	Informational	Log only, no intervention
WARNING	Potential issue	Adjust dose within safe limits
CRITICAL	Serious risk	Significant dose reduction
EMERGENCY	Immediate danger	Suspend insulin completely

1.6.5 4.5 Input Validation

The InputValidator checks:

```python
# Glucose range validation
if glucose < 20 or glucose > 600:
    raise InvalidGlucoseError(f"Glucose {glucose} outside valid range")

# Insulin request validation
if insulin < 0 or insulin > config.max_bolus:
    raise InvalidInsulinError(f"Insulin {insulin} invalid")

# Timestep validation
if timestep < 1 or timestep > 15:
    raise InvalidTimestepError(f"Timestep {timestep} invalid")
```

1.6.6 4.6 Simulation Termination

Automatic termination occurs when:

1. Critical hypoglycemia: Glucose < 40 mg/dL for 30+ minutes
2. Configuration error: Invalid safety configuration
3. Algorithm error: Unhandled exception in algorithm
4. Manual stop: User interrupts simulation

Termination Output: - `SimulationLimitError` exception raised - Safety report marks `terminated_early: true` - Final glucose and intervention reason logged - Partial results still saved

---

1.7 5. Tutorials and Cookbook

1.7.1 5.1 24-Hour Simulation Walkthrough

Complete example from setup to analysis:

```python
import iints
import pandas as pd
import matplotlib.pyplot as plt
from iints.core.algorithms.pid_controller import PIDController
from iints.core.simulator import Simulator, StressEvent

# 1. Setup simulation
sim = Simulator(
    algorithm=PIDController(),
    patient_config="default",
    time_step=5,  # 5-minute steps
    enable_profiling=True
)

# 2. Add realistic stress events
sim.add_stress_event(StressEvent(start_time=480, event_type="meal", value=60))  # 8:00 AM
 ↪  breakfast
sim.add_stress_event(StressEvent(start_time=720, event_type="meal", value=45))  # 12:00
 ↪  PM lunch
sim.add_stress_event(StressEvent(start_time=1080, event_type="exercise", value=45))  #
 ↪  6:00 PM workout
sim.add_stress_event(StressEvent(start_time=1320, event_type="meal", value=75))  # 8:00
 ↪  PM dinner

# 3. Run 24-hour simulation
results_df, safety_report = sim.run_batch(duration_minutes=1440)

# 4. Analyze results
print(f"Time in Range (70-180 mg/dL): {iints.metrics.calculate_tir(results_df):.1f}%")
print(f"Glucose Management Indicator: {iints.metrics.calculate_gmi(results_df):.1f}")
print(f"Safety interventions: {safety_report['intervention_count']}")

# 5. Visualize
plt.figure(figsize=(12, 6))
plt.plot(results_df['timestamp'], results_df['glucose_actual_mgdl'])
plt.axhline(180, color='red', linestyle='--', label='Hyperglycemia')
plt.axhline(70, color='green', linestyle='--', label='Target')
plt.axhline(54, color='orange', linestyle='--', label='Hypoglycemia')
plt.title('24-Hour Glucose Profile')
plt.xlabel('Time')
plt.ylabel('Glucose (mg/dL)')
plt.legend()
plt.grid(True)
plt.show()

# 6. Generate report
iints.generate_clinical_report(
    results_df,
    safety_report,
    "results/24hour_report.pdf"
)
```

1.7.2 5.2 Building an ML-Hybrid Algorithm

Combine ML prediction with rule-based safety:

```python
from iints.api.base_algorithm import InsulinAlgorithm
from iints.research.predictor import load_predictor_service

class MLHybridAlgorithm(InsulinAlgorithm):
    def __init__(self, predictor_path="models/predictor.pt"):
        super().__init__()
        self.predictor = load_predictor_service(predictor_path)
        self.last_prediction = None

    def predict_insulin(self, data: AlgorithmInput) -> dict:
        # 1. Get ML prediction (30-min forecast)
        prediction = self.predictor.predict(data)
        self.last_prediction = prediction['glucose_forecast']

        # 2. Rule-based decision with ML insight
        basal = 0.9  # U/hr
        bolus = 0.0

        # 3. Adjust based on prediction
        if prediction['glucose_forecast'] > 200:
            # Aggressive correction if rising
            bolus = (prediction['glucose_forecast'] - 180) / 40
        elif prediction['glucose_forecast'] < 90:
            # Conservative if dropping
            basal = max(0.3, basal * 0.7)  # Reduce basal but don't suspend

        return {
            "basal_insulin": basal,
            "bolus_insulin": bolus,
            "reason": f"ML forecast: {prediction['glucose_forecast']:.0f}mg/dL"
        }
```

1.7.3 5.3 Running Batch Experiments

Test multiple configurations efficiently:

```python
from iints.analysis.batch import run_batch_experiment

configurations = [
    {"algorithm": "PIDController", "patient": "default", "scenario": "baseline"},
    {"algorithm": "PIDController", "patient": "adolescent", "scenario":
↪   "meal_challenge"},
    {"algorithm": "MyAlgorithm", "patient": "default", "scenario": "baseline"},
    {"algorithm": "MyAlgorithm", "patient": "adolescent", "scenario": "meal_challenge"},
]

results = run_batch_experiment(
    configurations=configurations,
    duration_minutes=1440,
    output_dir="results/batch_experiment",
```

```
    parallel_workers=4  # Use 4 CPU cores
)

# Compare metrics across all runs
comparison_df = results.compare_metrics()
print(comparison_df[['algorithm', 'patient', 'TIR', 'GMI', 'interventions']])
```

1.7.4 5.4 Audit Trail Analysis

Every run produces a structured audit trail that explains why the safety layer intervened.

```
import pandas as pd

audit = pd.read_csv("results/your_run/audit/audit_trail.csv")
print(audit[['timestamp', 'glucose_actual_mgdl', 'action', 'reason']].head())

# Count interventions by type
print(audit['action'].value_counts())
```

Interpretation tips: - If `action` is `suspend` or `cap`, the supervisor overrode the algorithm. - If `reason` repeats often, your algorithm is too aggressive for that patient profile.

1.7.5 5.5 Custom Safety Thresholds

Use SafetyConfig to tighten or relax constraints for research experiments.

```
from iints.core.safety import SafetyConfig
from iints.core.simulator import Simulator

safe_config = SafetyConfig(
    max_bolus=3.0,
    max_iob=8.0,
    hyper_cutoff=250.0,
)

sim = Simulator(algorithm=MyAlgorithm(), safety_config=safe_config)
```

1.7.6 5.6 Pump Emulator Benchmarking

Benchmark alternative pump behaviors or commercial-emulator presets.

```
from iints.analysis.hardware_benchmark import benchmark_pump_emulators

bench = benchmark_pump_emulators(duration_minutes=120)
print(bench[['model', 'avg_step_ms', 'max_step_ms']])
```

1.7.7 5.7 Live Streaming Simulation

Stream real-time values for dashboards or demos.

```python
from iints.core.simulator import Simulator

sim = Simulator(algorithm=MyAlgorithm())
for state in sim.run_stream(duration_minutes=120):
    print(state['timestamp'], state['glucose_actual_mgdl'])
```

1.7.8 5.8 Reproducible Runs for Publications

To make results reproducible: - Fix `seed` - Persist `config.json` - Record dataset hash and commit SHA

```python
results = iints.run_simulation(
    algorithm=MyAlgorithm(),
    duration_minutes=720,
    seed=123,
)
print(results['run_manifest'])
```

1.8 6. API Reference

1.8.1 6.1 Core Classes

1.8.1.1 InsulinAlgorithm (Abstract Base Class)

```python
from iints.api.base_algorithm import InsulinAlgorithm, AlgorithmInput, AlgorithmResult

class MyAlgorithm(InsulinAlgorithm):
    def get_algorithm_metadata(self) -> dict:
        """Return algorithm identification and version info"""
        return {
            "name": "MyAlgorithm",
            "version": "1.0.0",
            "description": "My custom insulin dosing algorithm",
            "author": "Your Name",
            "reference": "Optional citation or paper reference"
        }

    def predict_insulin(self, data: AlgorithmInput) -> dict:
        """
        Calculate insulin dose based on current data

        Args:
            data: AlgorithmInput containing current state

        Returns:
            dict with keys: basal_insulin, bolus_insulin, reason
        """
        # Your algorithm logic here
        return {
            "basal_insulin": 0.9,  # U/hr
            "bolus_insulin": 0.0,  # U
```

```
            "reason": "Stable glucose, maintaining basal rate"
        }

    def reset(self):
        """Reset algorithm state for new simulation"""
        pass
```

1.8.1.2 AlgorithmInput (Dataclass)

```
@dataclass
class AlgorithmInput:
    # Current state
    current_glucose: float  # mg/dL
    current_time: datetime  # Simulation timestamp

    # Historical context
    glucose_history: List[float]  # Last 24 hours (5-min intervals)
    insulin_history: List[float]  # Last 24 hours
    carb_history: List[float]  # Last 24 hours

    # Calculated values
    iob: float  # Insulin on board (U)
    cob: float  # Carbs on board (g)

    # Trends
    glucose_trend: float  # mg/dL per 5 minutes
    glucose_acceleration: float  # mg/dL per 5 minutes^2

    # Patient info
    patient_config: dict  # ISF, ICR, basal rates, etc.

    # Safety context
    last_safety_intervention: Optional[dict]  # Last intervention reason
```

1.9 7. Practical Examples

1.9.1 7.1 Complete Algorithm Example

Full working algorithm with comprehensive logic:

```
from iints.api.base_algorithm import InsulinAlgorithm, AlgorithmInput
import numpy as np

class ComprehensiveAlgorithm(InsulinAlgorithm):
    def __init__(self):
        super().__init__()
        self.target_glucose = 120  # mg/dL
        self.isf = 50  # Insulin sensitivity factor
        self.icr = 10  # Insulin-to-carb ratio
        self.basal_rate = 0.9  # U/hr
        self.history = []
```

```python
    def get_algorithm_metadata(self):
        return {
            "name": "Comprehensive Algorithm",
            "version": "1.0.0",
           "description": "Full-featured algorithm with meal detection and trend analysis"
        }

    def predict_insulin(self, data: AlgorithmInput) -> dict:
        # Store history for trend analysis
        self.history.append(data.current_glucose)
        if len(self.history) > 24:
            self.history.pop(0)

        # Calculate correction bolus
        correction = 0.0
        if data.current_glucose > self.target_glucose:
            correction = (data.current_glucose - self.target_glucose) / self.isf

        # Meal detection (simple version)
        meal_bolus = 0.0
        if data.carb_history and data.carb_history[-1] > 0:
            meal_bolus = data.carb_history[-1] / self.icr

        # Trend adjustment
        trend_adjustment = 0.0
        if data.glucose_trend > 2:  # Rising fast
            correction *= 1.2  # More aggressive
        elif data.glucose_trend < -2:  # Dropping fast
            correction *= 0.8  # More conservative

        # IOB safety
        if data.iob > 5:  # High IOB
            self.basal_rate = max(0.3, self.basal_rate * 0.8)

        # Final dose calculation
        basal = self.basal_rate
        bolus = correction + meal_bolus

        return {
            "basal_insulin": basal,
            "bolus_insulin": bolus,
            "reason": (
                f"Target {self.target_glucose}mg/dL, "
                f"correction {correction:.2f}U, "
                f"meal {meal_bolus:.2f}U, "
                f"trend {data.glucose_trend:.1f}mg/dL per 5min"
            )
        }

    def reset(self):
        self.history = []
```

1.9.2 7.2 CLI vs Python API Comparison

Same Task: Run Simulation with Custom Algorithm

CLI Version:

```
# Create algorithm
iints new-algo --name MyAlgorithm --output-dir algorithms/

# Edit algorithm file
nano algorithms/my_algorithm.py

# Run simulation
iints run \
  --algo algorithms/my_algorithm.py \
  --patient-config-name default \
  --scenario-path scenarios/meal_challenge.json \
  --duration 1440 \
  --output-dir results/cli_run
```

Python Version:

```
from iints import run_simulation
from iints.core.algorithms.custom import MyAlgorithm

results = run_simulation(
    algorithm=MyAlgorithm(),
    patient_config="default",
    scenario="scenarios/meal_challenge.json",
    duration_minutes=1440,
    output_dir="results/python_run"
)
```

When to Use CLI: - Quick testing and iteration - Batch processing multiple scenarios - Integration with shell scripts - CI/CD pipelines

When to Use Python API: - Complex algorithm development - Integration with other Python tools - Custom analysis pipelines - Jupyter notebook exploration

1.9.3 7.3 Stress Testing Patterns

Pattern 1: Meal Challenge Test

```
# Test algorithm response to large meal
sim.add_stress_event(StressEvent(
    start_time=480,  # 8:00 AM
    event_type="meal",
    value=100  # 100g carbs (large pizza meal)
))
```

Pattern 2: Exercise Stress Test

```
# Test algorithm response to exercise
sim.add_stress_event(StressEvent(
    start_time=1080,  # 6:00 PM
    event_type="exercise",
    value=60  # 60 minutes intense exercise
))
```

Pattern 3: Sensor Noise Test

```
# Test algorithm robustness to CGM noise
sim.add_stress_event(StressEvent(
    start_time=300,  # 5:00 AM
    event_type="sensor_noise",
    value=20.0  # 20 mg/dL standard deviation
))
```

Pattern 4: Combined Stress Test

```
# Realistic day with multiple stressors
sim.add_stress_event(StressEvent(420, "meal", 45))  # 7:00 AM breakfast
sim.add_stress_event(StressEvent(660, "meal", 60))  # 11:00 AM snack
sim.add_stress_event(StressEvent(900, "exercise", 30))  # 3:00 PM walk
sim.add_stress_event(StressEvent(1020, "meal", 80))  # 5:00 PM dinner
sim.add_stress_event(StressEvent(1200, "sensor_noise", 15.0))  # 8:00 PM sensor issues
```

1.9.4 7.4 Human-in-the-Loop Integration

Add manual interventions during simulation:

```
def human_in_loop_callback(context):
    """
    Called at each simulation step.
    Return dict with manual interventions or None.
    """
    # Example: Rescue carbs for hypoglycemia
    if context["glucose_actual_mgdl"] < 65:
        return {
            "additional_carbs": 15,  # 15g fast-acting carbs
            "note": "Human intervention: rescue carbs for hypo"
        }

    # Example: Manual bolus correction
    if context["glucose_actual_mgdl"] > 250:
        return {
            "additional_bolus": 1.5,  # 1.5U correction
            "note": "Human intervention: manual correction bolus"
        }

    # Example: Suspend insulin before exercise
    if context["time"] > 1080 and context["time"] < 1140:  # 6-7 PM
        return {
            "suspend_insulin": True,
```

```
            "note": "Human intervention: exercise suspension"
        }

    return None  # No intervention

# Use in simulator
sim = Simulator(
    algorithm=MyAlgorithm(),
    patient_config="default",
    on_step=human_in_loop_callback
)
```

1.9.5 7.5 Data Import Workflows

Workflow 1: Dexcom CSV Import

```
# CLI import
iints import-data \
  --input-csv dexcom_export.csv \
  --data-format dexcom \
  --output-dir results/dexcom_import

# Use imported scenario
iints run \
  --algo algorithms/my_algorithm.py \
  --scenario-path results/dexcom_import/scenario.json
```

Workflow 2: Nightscout Import

```
# Install Nightscout extra
pip install iints-sdk-python35[nightscout]

# Import from Nightscout
iints import-nightscout \
  --url https://your-nightscout-site.herokuapp.com \
  --days 7 \
  --output-dir results/nightscout_import
```

Workflow 3: Dataset Registry

```
# List available datasets
iints data list

# Fetch specific dataset
iints data fetch aide_t1d --output-dir data_packs/aide

# Use in simulation
iints run \
  --algo algorithms/my_algorithm.py \
  --scenario-path data_packs/aide/scenarios/patient_001.json
```

1.10 8. Advanced Topics

1.10.1 8.1 Commercial Pump Emulators

Test against real pump behavior:

```python
from iints.emulation import Medtronic780G, Omnipod5, TandemControlIQ

# Compare your algorithm against commercial pumps
pumps = [
    ("MyAlgorithm", MyAlgorithm()),
    ("Medtronic 780G", Medtronic780G()),
    ("Omnipod 5", Omnipod5()),
    ("Tandem Control-IQ", TandemControlIQ())
]

results = {}
for name, pump_algo in pumps:
    results[name] = run_simulation(
        algorithm=pump_algo,
        patient_config="default",
        duration_minutes=1440
    )

# Compare metrics
compare_metrics(results)
```

1.10.2 8.2 Dataset Registry Usage

Access real-world datasets:

```bash
# List all available datasets
iints data list

# Get info about specific dataset
iints data info aide_t1d

# Fetch dataset
iints data fetch aide_t1d --output-dir data_packs/aide

# Cite dataset in publication
iints data cite aide_t1d
```

Available Datasets: - `aide_t1d`: AIDE Type 1 Diabetes Dataset - `azt1d`: Arizona Type 1 Diabetes Dataset - `ohio_t1dm`: OhioT1DM Dataset - `sample`: Bundled demo data (no download needed)

Integrity and reproducibility - Every dataset entry includes a SHA-256 checksum and citation metadata. - The fetch command validates the checksum automatically. - Use `iints data info <dataset>` to record version and hash in your paper.

Typical layout after fetch

```
data_packs/
  public/
```

```
    ohio_t1dm/
      raw/...
      processed/...
```

1.10.3 8.3 Reproducibility Techniques

Ensure identical results across runs:

```python
# Method 1: Set random seed
results = run_simulation(
    algorithm=MyAlgorithm(),
    seed=42,  # Fixed random seed
    patient_config="default"
)

# Method 2: Use deterministic patient profile
profile = PatientProfile(
    isf=50,
    icr=10,
    basal_rate=0.9,
    dawn_phenomenon=0.3,  # Fixed dawn effect
    seed=42
)

# Method 3: SHA-256 verification
from iints.utils.hashing import verify_manifest

if verify_manifest("results/run_001/run_manifest.json"):
    print("Results verified - not tampered with")
```

1.10.4 8.4 Performance Profiling

Measure algorithm performance:

```python
sim = Simulator(
    algorithm=MyAlgorithm(),
    patient_config="default",
    enable_profiling=True
)

results_df, safety_report = sim.run_batch(duration_minutes=1440)

# Access performance data
profiling = safety_report["performance_report"]
print(f"Algorithm latency: {profiling['algorithm_latency_ms']:.2f}ms")
print(f"Safety latency: {profiling['safety_latency_ms']:.2f}ms")
print(f"Total steps: {profiling['total_steps']}")
print(f"Total time: {profiling['total_time_s']:.2f}s")
```

1.10.5 8.5 Custom Metrics Calculation

Define your own performance metrics:

```python
def calculate_custom_metric(results_df):
    """Calculate custom performance metric"""

    # Time in tight range (80-140 mg/dL)
    tight_range = ((results_df['glucose_actual_mgdl'] >= 80) &
                   (results_df['glucose_actual_mgdl'] <= 140)).mean() * 100

    # Glucose variability score
    cv = results_df['glucose_actual_mgdl'].std() /
         results_df['glucose_actual_mgdl'].mean() * 100

    # Hypoglycemia risk score
    hypo_risk = (results_df['glucose_actual_mgdl'] < 70).sum() / len(results_df)

    # Hyperglycemia risk score
    hyper_risk = (results_df['glucose_actual_mgdl'] > 250).sum() / len(results_df)

    # Composite score (lower is better)
    composite_score = (100 - tight_range) + cv + (hypo_risk * 100) + (hyper_risk * 50)

    return {
        'tight_range_percent': tight_range,
        'cv_percent': cv,
        'hypo_risk_percent': hypo_risk * 100,
        'hyper_risk_percent': hyper_risk * 100,
        'composite_score': composite_score
    }

# Use with your results
metrics = calculate_custom_metric(results_df)
print(f"Custom Score: {metrics['composite_score']:.1f}")
```

1.10.6 8.6 Local AI Assistant (Ministral 3 Open-Weight via Ollama)

The SDK includes a research-only local AI assistant for simulation explanation and summary generation.

What it is for: - Explain a single decision step in plain language - Summarize glycemic trends from a run payload - Detect anomalies in a result summary - Generate a short markdown run report

What it is not for: - Clinical dosing advice - Live patient-facing recommendations - Medical decision support

Safety gate before any LLM call

Every AI command is blocked unless the MDMP artifact verifies successfully and meets the minimum required grade.

The AI flow is: 1. Load simulation JSON from disk 2. Verify signed MDMP artifact with `MDMPGuard` 3. Check that Ollama is reachable 4. Resolve the requested local Ministral 3 model tag 5. Generate a response 6. Append a hard-coded research-only disclaimer

Recommended setup

Always use an active virtual environment:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -U pip
python -m pip install -e ".[mdmp]"
```

Pull and validate the local model:

```
python -m pip install -U "iints-sdk-python35[mdmp]"
iints ai models
ollama pull ministral-3:8b
iints ai local-check --model ministral-3:8b
```

`local-check` now includes a small generation smoke-test by default, so it verifies real model inference readiness and not just basic Ollama reachability.

Recommended run workflow

```
iints quickstart --project-name iints_quickstart
cd iints_quickstart
iints presets run --name baseline_t1d --algo algorithms/example_algorithm.py
iints ai prepare results/<run_id>
iints ai report results/<run_id>
```

`iints ai prepare` creates AI-ready JSON payloads in `results/<run_id>/ai/` and, when the MDMP extra is installed, also generates a local development MDMP certificate plus companion keypair so the assistant can run without hand-building `step.json` and `report.signed.mdmp`.

If local generation still disconnects, the first practical fallback is to switch from `ministral-3:8b` to `ministral-3:3b`.

Personal CareLink workflow

If you imported your own MiniMed / CareLink data, you can build a personal workspace that combines: - a standard IINTS timeline - a scenario for experiments - a visual dashboard - AI-ready payloads

```
iints carelink-workbench \
  --input-csv "/path/to/CareLink export.csv" \
  --output-dir results/personal_carelink
```

This creates: - `carelink_timeline.csv` - `carelink_metrics.json` - `carelink_dashboard.png` - `carelink_poster.png` - `carelink_dashboard.html` - `ai/report_payload.json` - `ai/trends_payload.json` - `ai/anomalies_payload.json` - `ai/step_riskiest.json`

If the MDMP extra is installed, the workbench also generates a local development certificate so the assistant can explain imported personal data without any manual JSON preparation.

Inference commands

Prepared run directory mode:

```
iints ai explain results/<run_id>
iints ai trends results/<run_id>
iints ai anomalies results/<run_id>
iints ai report results/<run_id> \
  --output results/<run_id>/ai/ai_report.md
```

Prepared personal CareLink workspace mode:

```
iints ai report results/personal_carelink --model ministral-3:3b
iints ai trends results/personal_carelink --model ministral-3:3b
iints ai explain results/personal_carelink --model ministral-3:3b
```

Direct JSON mode:

```
iints ai explain results/step.json \
  --mdmp-cert results/report.signed.mdmp

iints ai trends results/glucose_payload.json \
  --mdmp-cert results/report.signed.mdmp

iints ai anomalies results/simulation_run.json \
  --mdmp-cert results/report.signed.mdmp

iints ai report results/simulation_run.json \
  --mdmp-cert results/report.signed.mdmp \
  --output results/ai_report.md
```

Operational notes - The default local model is `ministral-3:8b`. - Friendly aliases like `ministral` are resolved automatically against installed Ollama tags. - Older `Ministral 8B` tags remain accepted as backward-compatible fallbacks if they are already installed locally. - Users can select a different local model with `--model`, for example `ministral-3:3b` or `ministral-3:14b`.

PC specification guidance

Model	Best fit	Recommended system RAM	Recommended GPU VRAM	Approx download
`ministral-3:3b`	Smaller laptop / CPU-first setup	16 GB	6 GB	~3 GB
`ministral-3:8b`	Balanced desktop / strong laptop	24 GB	10 GB	~6 GB
`ministral-3:14b`	High-end workstation	32 GB	16 GB	~10 GB

Suggested rule: - start with `ministral-3:8b` - move down to `ministral-3:3b` if memory or latency is tight - move up to `ministral-3:14b` only if you have headroom and want better answer quality - Oversized JSON payloads are clipped automatically before prompt construction so local inference stays practical on smaller systems. - Use `--timeout-seconds 120` or higher for slower edge hardware.

1.11 11. Poster-Ready Results for Jury Demos

The SDK can now turn one to three completed run bundles into a single poster-style PNG. This is ideal for science fairs, thesis defenses, pitch decks, and jury presentations where you want three scenarios side by side.

Recommended story - Normal run - Meal stress test - Supervisor override / safety intervention

Command

```
iints poster \
  --run-dir results/normal_run \
  --run-dir results/meal_stress \
  --run-dir results/supervisor_override \
  --label "Normal Run" \
  --label "Meal Stress Test" \
  --label "Supervisor Override" \
  --output-path results/posters/iints_results_poster.png
```

What it includes - Glucose curve for each run - Target range highlight (`70–180 mg/dL`) - Meal markers when carbs are present - Supervisor intervention markers when the safety layer triggered - Per-panel summary box with TIR, time below range, meal count, and intervention count

Outputs - `results/posters/iints_results_poster.png` - `results/posters/iints_results_poster.json`

If you omit `--run-dir`, the SDK auto-discovers the latest run bundles under `./results`.

1.12 12. Booth / Jury Demo Bundle

If you need a live demo for a science fair, jury room, or expo booth, the SDK now includes a one-command flow that builds the whole story for you.

Command

```
./scripts/run_booth_demo.sh
```

Or, if you want to stay inside the installed CLI:

```
iints demo-booth --output-dir results/booth_demo
```

What it creates - `Normal Run` - `Meal Stress Test` - `Supervisor Override` - a combined poster PNG - a markdown talk track for the jury - optional AI-ready artifacts for the safety scenario

Main outputs - `examples/demos/07_live_stage_demo.py` - `results/booth_demo/booth_demo_poster.png` - `results/booth_demo/JURY_TALK_TRACK.md` - `results/booth_demo/run_commands.md` - `results/booth_demo/demo_summary.json`

Best live flow

For a booth or jury table, the cleanest explanation is:

1. Open `examples/demos/07_live_stage_demo.py`.
2. Point to `PATIENT_CONFIG`, `OUTPUT_DIR`, `DURATION_MINUTES`, `TIME_STEP_MINUTES`, and `SEED`.
3. Point out that the script visibly calls `run_full(...)`, `generate_results_poster(...)`, and `prepare_ai_ready_artifacts(...)`.
4. Explain that swapping the patient profile reruns the same pipeline for another patient.
5. Run:

```
./scripts/run_live_stage_demo.sh
```

6. Open:

- `results/booth_demo_live/booth_demo_poster.png`
- `results/booth_demo_live/JURY_TALK_TRACK.md`
- `results/booth_demo_live/BEURS_LIVE_DEMO_SCRIPT.txt`

7. If the machine only has the installed SDK and not the repository checkout, export the same code first:

```
iints demo-export --output-dir iints_demo
cd iints_demo
python 07_live_stage_demo.py
```

8. If someone wants more proof, open a scenario folder and show `results.csv`, `clinical_report.pdf`, and `run_manifest.json`.

Why this is useful - You get real run bundles, not hand-built presentation images. - You can explain normal control, stress handling, and safety intervention in one pass. - The audience can see that the SDK is reproducible, visual, and safety-first.

Troubleshooting - If `iints ai local-check` fails, first confirm Ollama is running. - If Ollama is reachable but the model is missing, run the exact `ollama pull ...` command shown by the SDK. - If generation is slow, increase `--timeout-seconds` and use a smaller JSON payload.

1.13 9. Troubleshooting

1.13.1 9.1 Installation Issues

Issue: ModuleNotFoundError after installation

Solution:

```
# Make sure you're using the correct Python environment
which python  # Should show your virtual environment path

# Reinstall in development mode if needed
pip install -e .
```

Issue: pip install fails with dependency errors

Solution:

```
# Upgrade pip first
pip install --upgrade pip setuptools wheel

# Try installing with --no-cache-dir
pip install --no-cache-dir iints-sdk-python35

# For specific errors, check Python version
python3 --version  # Must be 3.10+
```

Issue: another machine still behaves like an old SDK after upgrading

Solution:

```
source .venv/bin/activate
python -m pip install -U pip
python -m pip install -U "iints-sdk-python35[mdmp]"
hash -r
python -c "import iints; print(iints.__version__)"
iints-sdk-doctor
```

If `iints-sdk-doctor` reports a conflict:

```
python -m pip uninstall -y iints iints-sdk-python35
python -m pip install -U "iints-sdk-python35[mdmp]"
hash -r
```

For the full upgrade walkthrough, see `docs/UPDATING.md`. By default, the upgrade commands in that guide track the latest release. Only pin an exact version when you need reproducible packaging for a demo, paper, or audit.

1.13.2 9.2 Simulation Issues

Issue: Simulation runs very slowly

Solution:

```
# Increase time step (default is 5 minutes)
sim = Simulator(time_step=10)  # 10-minute steps

# Disable profiling if not needed
sim = Simulator(enable_profiling=False)

# Reduce duration for testing
results = sim.run_batch(duration_minutes=720)  # 12 hours instead of 24
```

Issue: Simulation terminates early

Solution:

```
# Check safety report for termination reason
print(safety_report['termination_reason'])

# Common causes:
# - Critical hypoglycemia (< 40 mg/dL for 30+ minutes)
# - Algorithm exception
# - Configuration error

# Adjust safety limits if needed
from iints.core.safety import SafetyConfig
config = SafetyConfig(critical_hypo_cutoff=35.0)  # Lower threshold
```

1.13.3 9.3 Algorithm Development Issues

Issue: Algorithm not appearing in CLI

Solution:

```
# Make sure algorithm is properly registered
# 1. Inherits from InsulinAlgorithm
# 2. Implements all required methods
# 3. Has valid get_algorithm_metadata()

# Check with:
from iints.api.registry import list_algorithms
print(list_algorithms())
```

Issue: Safety supervisor blocking all doses

Solution:

```
# Check safety configuration
print(safety_report['config'])

# Common issues:
# - max_basal_rate too low
# - hypo_cutoff too high
# - contract enabled with aggressive settings

# Adjust configuration:
config = SafetyConfig(
    hypo_cutoff=60.0,  # Higher threshold
    max_basal_rate=1.5  # Higher limit
)
```

1.13.4 9.4 Data Import Issues

Issue: CSV import fails

Solution:

```
# Check CSV format
# Required columns: timestamp, glucose
# Optional: carbs, insulin

# Try auto-detect format
data = import_cgm_csv("your_file.csv", data_format="auto")

# Specify column names manually if needed
data = import_cgm_csv(
    "your_file.csv",
    data_format="generic",
    timestamp_col="Date",
    glucose_col="BG",
    carbs_col="Carbs"
)
```

1.13.5 9.5 Performance Issues

Issue: High memory usage

Solution:

```
# Reduce history size
sim = Simulator(max_history_hours=12)  # Default is 24

# Disable audit trail if not needed
sim = Simulator(enable_audit=False)

# Process in batches
for i in range(10):
    results = sim.run_batch(duration_minutes=144)  # 2.4 hours per batch
    process_results(results)
```

1.14 10. Quick Reference

1.14.1 10.1 Essential CLI Commands

```
# Initialize project
iints init --project-name my_project

# Quickstart
iints quickstart --project-name demo

# Run simulation
iints run --algo algorithms/my_algo.py --duration 1440

# Run with preset
iints presets run --name baseline_t1d --algo algorithms/my_algo.py

# Benchmark
```

```
iints benchmark --algo-to-benchmark algorithms/my_algo.py

# Import data
iints import-data --input-csv my_cgm.csv --data-format dexcom

# List presets
iints presets list

# Generate scenario
iints scenarios generate --name "Stress Test"

# Validate files
iints validate --scenario-path scenarios/my_scenario.json

# Check local Ministral backend
iints ai local-check --model ministral

# Prepare a finished run for AI
iints ai prepare results/<run_id>

# Generate AI explanation
iints ai explain results/<run_id>
```

1.14.2 10.2 Python Code Snippets

Minimal Simulation:

```python
from iints import run_simulation
from iints.core.algorithms.pid_controller import PIDController

results = run_simulation(
    algorithm=PIDController(),
    duration_minutes=720
)
```

Custom Algorithm:

```python
from iints.api.base_algorithm import InsulinAlgorithm

class MyAlgorithm(InsulinAlgorithm):
    def predict_insulin(self, data):
        return {"basal_insulin": 0.9, "bolus_insulin": 0.0}
```

Load Results:

```python
import pandas as pd
df = pd.read_csv(results['results_csv'])
print(df[['timestamp', 'glucose_actual_mgdl', 'insulin_delivered']].head())
```

1.14.3 10.3 Safety Thresholds (Defaults)

```
SafetyConfig(
    hypo_cutoff=70.0,          # Start reducing insulin
    severe_hypo_cutoff=54.0,   # Emergency suspension
    critical_hypo_cutoff=40.0, # Terminate simulation
    hyper_cutoff=300.0,        # Maximum glucose
    max_basal_rate=2.0,        # U/hr
    max_bolus=5.0,             # U per bolus
    max_iob=10.0,              # U total active insulin
    max_insulin_per_hour=15.0, # U/hr rolling window
    max_insulin_per_day=80.0   # U/day absolute limit
)
```

1.14.4 10.4 Clinical Metrics Targets (ATTD/ADA)

Metric	Target Range
TIR (70-180 mg/dL)	>70%
TBR (<70 mg/dL)	<4%
TBR (<54 mg/dL)	<1%
GMI	<7.0%
CV	<36%
LBGI	<1.1
HBGI	<5.0

1.15 11. Glossary

Algorithm: Code that calculates insulin doses based on current and historical data

Basal Insulin: Background insulin delivery rate (U/hr)

Bolus Insulin: Additional insulin for meals or corrections (U)

CGM: Continuous Glucose Monitor - device that measures glucose every 5 minutes

COB: Carbs On Board - carbohydrates still being absorbed

GMI: Glucose Management Indicator - estimate of A1C from CGM data

IOB: Insulin On Board - insulin still active in the body

ISF: Insulin Sensitivity Factor - how much 1U of insulin lowers glucose (mg/dL)

ICR: Insulin-to-Carb Ratio - grams of carbs covered by 1U of insulin

TIR: Time In Range - percentage of time in target glucose range (70-180 mg/dL)

TBRL1: Time Below Range Level 1 - percentage of time <70 mg/dL

TBRL2: Time Below Range Level 2 - percentage of time <54 mg/dL

TAR: Time Above Range - percentage of time >180 mg/dL

CV: Coefficient of Variation - measure of glucose variability

LBGI: Low Blood Glucose Index - measure of hypoglycemia risk

HBGI: High Blood Glucose Index - measure of hyperglycemia risk

---

1.16 Need More Help?

Documentation: - Comprehensive Guide: `docs/COMPREHENSIVE_GUIDE.md` - Technical README: `docs/TECHNICAL_README.md` - API Stability: `API_STABILITY.md`

Community: - GitHub Issues: https://github.com/python35/IINTS-SDK/issues - Discussion Forum: [Link] - Email Support: support@iints.org

Citing IINTS-AF:

```
@software{IINTS_AF_SDK,
  author = {Bobbaers, Rune},
  title = {IINTS-AF: Intelligent Insulin Titration System for Artificial Pancreas},
  year = {2026},
  version = {0.1.22},
  url = {https://github.com/python35/IINTS-SDK},
  note = {Pre-clinical research software for insulin dosing algorithm validation}
}
```

PRE-CLINICAL USE ONLY - NOT FOR PATIENT CARE

This document is provided for research and educational purposes only. The IINTS-AF SDK is not a medical device and should not be used for clinical decision-making.